

Previous solve – 18-19

1(a) RAID Working Mechanism

RAID (Redundant Array of Independent/Inexpensive Disks)

- **English:** RAID combines multiple disks to improve **performance** and/or **fault tolerance**.
- **Bangla:** একাধিক হার্ড ডিস্ক একসাথে যুক্ত করে ডেটা রাখা হয় যাতে **দ্রুত পড়া/লেখা** সম্ভব হয় এবং **ডেটা হারানো কম হয়**।

Working Mechanism:

1. **Data Striping:** Data কে ছোট ছোট block এ ভাগ করে multiple disks এ রাখে → **speed বৃদ্ধি**
2. **Mirroring:** একই data একাধিক disk এ রাখে → **fault tolerance বৃদ্ধি**
3. **Parity:** Extra parity bit/blocks ব্যবহার করে **error detection & recovery**

Bangla Summary:

- Striping → **দ্রুত**
 - Mirroring → **নিরাপদ**
 - Parity → **ভুল ধরার ও ঠিক করার জন্য**
-

1(b) Bus Performance & Types

Why performance suffers with many devices:

- **English:** If many devices share a bus, **each device waits for its turn** → **slower communication**.
- **Bangla:** অনেক device একসাথে bus use করলে, প্রতিটা device **পরপর অপেক্ষা** করতে হয় → **system ধীর হয়ে যায়**।

Types of Bus

Bus Type	Function	Bangla Explanation
Local Bus	Connects CPU directly with high-speed devices (like RAM, graphics)	CPU কে fast device সাথে যুক্ত করে, speed বেশি
System Bus	Connects CPU, memory, and I/O devices	CPU, memory, I/O device এর মূল connection
Expansion Bus	Connects slower devices via slots (PCI, ISA, USB)	Extra devices যেমন printer, scanner connect করতে use হয়, speed কম

1(c) Characteristics of RISC & CISC

Feature	RISC	CISC
Instruction Set	Simple, few instructions	Complex, many instructions
Execution	Usually 1 cycle per instruction	Multiple cycles per instruction
Instruction Length	Fixed	Variable
Hardware	Simple	Complex
Example	ARM, MIPS	Intel x86

Bangla Summary:

- **RISC:** ছোট, simple, fast, কম hardware
- **CISC:** বড়, জটিল, slow, বেশি hardware

1(d) Set Associative Mapping & Replacement Algorithms

Set Associative Mapping:

- **English:** Memory block $\rightarrow$ specific set in cache $\rightarrow$ any block in that set.
- **Bangla:** Cache কে multiple sets এ ভাগ করা হয়, main memory block ঠিক set এ যায়, কিন্তু set এর মধ্যে যেকোনো block রাখা যায়।

Example:

- Cache = 8 blocks, 2-way set associative $\rightarrow$ 4 sets $\times$ 2 blocks/set

- Memory block → maps to one set, choose 1 block from 2 to replace

Replacement Algorithms:

1. **FIFO (First In First Out):** পুরানো block replace
2. **LRU (Least Recently Used):** দীর্ঘ সময় ধরে ব্যবহার না হওয়া block replace
3. **LFU (Least Frequently Used):** সবচেয়ে কম ব্যবহার হওয়া block replace
4. **Random Replacement:** Randomly block replace

Bangla Summary:

- FIFO → সহজ, কখনো inefficient
 - LRU → recent use consider করে
 - LFU → frequent use consider করে
 - Random → random replace, সবসময় best নয়
-

Quick Tip:

- RAID = **speed + safety**
 - Bus = **connection + speed**
 - RISC vs CISC = **simple vs complex**
 - Cache mapping & replacement = **set → select block → replace algorithm**
-

2(a) OS working mechanism (User ↔ Hardware)

- **English:** Operating System (OS) acts as an **interface between user and hardware**.
- **Bangla:** OS হলো **মধ্যবর্তী layer**, যা ব্যবহারকারীর command কে hardware বুঝতে সাহায্য করে এবং hardware এর result ব্যবহারকারীর কাছে পৌঁছে দেয়।

Working Mechanism:

1. User gives command (keyboard, mouse, GUI)
2. OS interprets command → converts to **hardware instructions**
3. Hardware executes the instructions (CPU, memory, I/O devices)

4. Result sent back to user via OS

Bangla summary:

- User → OS → Hardware → OS → User
-

2(b) Two's complement division: divisor = -3, dividend = -7

Step 1: Represent numbers in 4-bit (example)

- $3 = 0011 \rightarrow -3$ (Two's complement) = 1101
- $7 = 0111 \rightarrow -7$ (Two's complement) = 1001

Step 2: Division rule (Two's complement)

- Sign of quotient = sign of dividend XOR sign of divisor
- Sign of remainder = same as dividend

Step 3: Calculate absolute division

- $|7| \div |3| \rightarrow \text{Quotient} = 2, \text{Remainder} = 1$

Step 4: Apply signs

- Dividend = -7, Divisor = -3 $\rightarrow$ quotient = +2, remainder = -1

 Answer:

- Quotient = 2
 - Remainder = -1
-

2(c) Random Access Memory (RAM) & Number Representations

RAM definition

- **English:** RAM = Memory which can be read and written in any order.
- **Bangla:** RAM হলো এমন memory যা থেকে যেকোনো address থেকে data পড়া বা লেখা যায়, sequential এর প্রয়োজন নেই।

Two's complement vs Sign magnitude

Feature	Two's Complement	Sign Magnitude
Representation of negative	Invert + 1	MSB = sign bit (1 = negative, 0 = positive)
Arithmetic	Easy (addition/subtraction same as positive)	Hard (need separate logic for subtraction)
Zero	Single zero	Two zeros (+0, -0)
Example: -5 (4-bit)	1011	1001

Bangla summary:

- Two's complement → calculation সহজ, এক zero
- Sign magnitude → calculation কঠিন, দুই zero

2(d) Direct Mapped Cache Problem

Given:

- Main memory = 2^{16} bytes
- Block size = 8 bytes → 3 bits for byte offset ($2^3 = 8$)
- Cache lines = 32 → 5 bits for line number ($2^5 = 32$)
- Tag bits = remaining → $16 - 5 - 3 = 8$ bits

2(d)(i) Memory address division

- **Tag** = 8 bits
- **Line number** = 5 bits
- **Byte number** = 3 bits

2(d)(ii) Cache line calculation

Memory address format:

[Tag (8)] [Line (5)] [Byte (3)]

1. Address = 0001 0001 0001 1011

- Tag = 00010001

- Line = 00011 → Decimal = 3
 - Byte = 011 → Decimal = 3
2. Address = 1100 0011 0011 0100
- Tag = 11000011
 - Line = 00110 → Decimal = 6
 - Byte = 100 → Decimal = 4

✅ **Answer:**

- First address → Line 3
- Second address → Line 6

2(d)(iii) Total cache bytes

- Cache lines = 32
 - Block size = 8 bytes
- [
 $\text{Total cache bytes} = 32 \times 8 = 256 \text{ bytes}$
]

💡 **Summary Table:**

Parameter	Value
Byte offset	3 bits
Line number	5 bits
Tag	8 bits
Line for 0001 0001 0001 1011	3
Line for 1100 0011 0011 0100	6
Total cache memory	256 bytes

ঠিক আছে! আমি তোমাকে সহজ বাংলায় + ইংরেজি মিশিয়ে step by step বোঝাচ্ছি।

3(a) Program-driven vs Interrupt-driven I/O

Feature	Program-driven I/O	Interrupt-driven I/O
CPU waiting	CPU continuously checks device (polling)	CPU is interrupted when device is ready
Efficiency	Low, wastes CPU cycles	High, CPU can do other tasks
Example	Loop waiting for keyboard input	Keyboard generates interrupt when key pressed
Bangla Summary	CPU সবসময় device কে check করে → slow	Device ready হলে interrupt দেয় → efficient

3(b)(i) Micro-operations to add two memory operands

Instruction: ADD M1, M2 (result stored in AC/Accumulator)

Steps:

1. Fetch instruction:

- $MAR \leftarrow PC$
- $MDR \leftarrow M[MAR]$
- $IR \leftarrow MDR$
- $PC \leftarrow PC + 1$

2. Fetch operands:

- $MAR \leftarrow \text{Address of M1}$
- $MDR \leftarrow M[MAR]$
- $AC \leftarrow MDR$ (AC stores first operand)
- $MAR \leftarrow \text{Address of M2}$
- $MDR \leftarrow M[MAR]$

3. Execute addition:

- $AC \leftarrow AC + MDR$

4. Store result (optional):

- $M[\text{Address of result}] \leftarrow AC$

Bangla Summary:

- Fetch $\rightarrow$ Operand load $\rightarrow$ Execute $\rightarrow$ Store
-

3(b)(ii) Two's complement addition (6-bit)

Given: $-1A + 2B$ (Hex)

Step 1: Convert Hex to binary (6-bit)

- $1A \text{ Hex} \rightarrow 011010 \rightarrow -1A \rightarrow \text{Two's complement} \rightarrow \text{invert} + 1 \rightarrow 100110$
- $2B \text{ Hex} \rightarrow 101011$

Step 2: Add binary numbers

100110 (-1A)
+ 101011 (2B)

010001 (Ignore carry beyond 6 bits)

Step 3: Convert result back to Hex

- $010001 = 11 \text{ Hex}$

✅ **Answer:** 11 Hex

3(c) I/O Module & External Device Classification

Why I/O module is used?

- **English:** To interface CPU with external devices.
- **Bangla:** CPU সরাসরি device handle করতে পারে না $\rightarrow$ I/O module CPU $\leftrightarrow$ device manage করে।

Functions:

1. Data transfer between CPU and device
2. Device control

3. Buffering
4. Error detection

External Device Classification:

Type	Examples	Bangla
Input	Keyboard, Mouse, Scanner	Data input
Output	Monitor, Printer, Speaker	Data output
Input/Output	Network card, Touchscreen	Data both ways

3(d) Short Notes

i. Dual-core

- **English:** Processor with **two cores** on a single chip → can execute **two tasks simultaneously**.
- **Bangla:** এক processor chip এ দুইটি core → একই সময়ে দুইটি কাজ করতে পারে → speed বৃদ্ধি।

ii. Pentium 4

- **English:** Intel CPU released in early 2000s, **high clock speed**, single-core, used **NetBurst architecture**.
- **Bangla:** পুরোনো Intel processor, দ্রুত clock, single core, performance-oriented।

iii. Core i7

- **English:** Modern Intel CPU, **multi-core**, hyper-threading, high performance for desktop & laptop.
 - **Bangla:** Intel এর নতুন generation, multiple cores + threads, high-speed processing, gaming & professional use।
-

Quick Tip:

- Program-driven = CPU polling → slow
- Interrupt-driven = CPU interrupted → fast

- I/O module → CPU ↔ devices interface
- Dual-core/Core i7 → modern multi-tasking, high speed
- Pentium 4 → older Intel CPU

4(a) Computer Organization vs Architecture & Main Structural Components

1. Computer Architecture vs Organization

Feature	Computer Architecture	Computer Organization
Definition	Logical design of a computer, what a computer does	Physical implementation, how it is built
Example	Instruction set, addressing modes	ALU design, memory circuits, bus structure
Bangla summary	Architecture = কম্পিউটার কি করে	Organization = কম্পিউটার কিভাবে কাজ করে

2. Main Structural Components of a Computer

1. **CPU (Central Processing Unit)** – Performs calculations and controls operations.
2. **Memory** – Stores data and instructions.
3. **Input Devices** – Provide data to the computer (keyboard, mouse).
4. **Output Devices** – Display results (monitor, printer).
5. **Control Unit (CU)** – Directs data flow between CPU, memory, and I/O.
6. **ALU (Arithmetic Logic Unit)** – Performs arithmetic & logic operations.
7. **Registers** – Small, fast storage inside CPU for temporary data.

Bangla summary:

- CPU + Memory + I/O + CU + ALU + Registers → computer এর main parts।

4(b) Moore's Law & IAS Memory Read/Write

1. Moore's Law

- **English:** The number of transistors on a chip **doubles approximately every 18–24 months**.
 - **Bangla:** একটি chip এ transistor সংখ্যা প্রায় প্রতি ১.৫–২ বছরে দ্বিগুণ হয়, → speed এবং performance বৃদ্ধি পায়।
-

2. IAS Memory Read/Write Process

Components:

- **MAR (Memory Address Register)** – Holds address to access memory
- **MBR (Memory Buffer Register)** – Holds data read from or to be written to memory
- **Address Bus** – Carries address from CPU to memory
- **Data Bus** – Carries data between CPU and memory
- **Control Bus** – Sends read/write signals

Memory Read:

1. MAR ← Address of memory
2. Control bus ← Read signal
3. Memory sends data → Data bus → MBR
4. CPU uses data from MBR

Memory Write:

1. MAR ← Address of memory
2. MBR ← Data to write
3. Control bus ← Write signal
4. Data bus sends data → memory

Bangla summary:

- Read → MAR address, Read signal, MBR gets data
 - Write → MAR address, MBR data, Write signal → memory
-

4(c) Key Properties of Semiconductor Memory & DRAM vs SRAM

1. Key Properties

1. **Volatility:** Loses data when power off (for RAM)
2. **Access time:** Speed to read/write data
3. **Density:** Number of bits per chip
4. **Cost:** Price per bit
5. **Power consumption**

2. DRAM vs SRAM

Feature	DRAM	SRAM
Storage	1 transistor + 1 capacitor per bit	6 transistors per bit
Speed	Slower	Faster
Cost	Cheaper	Expensive
Refresh	Needs refresh	No refresh
Use	Main memory	Cache memory

Bangla summary:

- DRAM → ধীরে, সস্তা, refresh লাগে
 - SRAM → দ্রুত, ব্যয়বহুল, refresh লাগে না
-

4(d) Memory Cell Definition & Structure

1. Memory Cell

- **English:** Smallest unit in memory that stores **1 bit of data**.
- **Bangla:** Memory cell হলো smallest unit যা **1 bit data রাখে**।

2. Typical Memory Cell Structure

- **SRAM cell:** 6 transistors forming a flip-flop
- **DRAM cell:** 1 transistor + 1 capacitor
- **Function:** Stores 0 or 1 and allows read/write via word line and bit line

Simple diagram idea (SRAM flip-flop):

Q ----|>----+

Q' ---|>----+

(cross-coupled inverters store 1 bit)

Bangla summary:

- SRAM → fast, transistor-based
 - DRAM → capacitor + transistor, needs refresh
-

💡 Quick Tip:

- Architecture = কি করে
- Organization = কিভাবে করা হয়
- Moore's Law → chip speed + transistor growth
- IAS read/write → MAR, MBR, buses, control signal
- Memory → DRAM (main), SRAM (cache), cell = 1 bit storage

5(a) Chip Logic & 16M × 4 DRAM Structure

1. What is Chip Logic?

- **English:** Chip logic refers to the **internal circuitry and logic gates on a memory or processor chip** that perform operations like storing, reading, and writing data.
 - **Bangla:** Chip logic হলো একটি chip-এর ভিতরের circuitry, যা data read/write, address decoding, এবং control signals পরিচালনা করে।
-

2. 16 Megabit DRAM (4M × 4) Structure

- **16 Megabit DRAM** → 16×2^{20} bits = 16,777,216 bits
- Organization: **4M × 4**
 - $4M = 4 \times 2^{20} = 4,194,304$ words
 - Each word = 4 bits → $4 \times 4,194,304 = 16,777,216$ bits

Structure:

- Memory divided into **rows and columns** (2D array)

- **Row decoder + Column decoder** selects specific word
- **4-bit data bus** transfers data to/from CPU
- Each cell = **1 transistor + 1 capacitor** (DRAM)

Bangla summary:

- 16Mbit = 4M words × 4 bits
- Row & column decoder selects word
- Data bus = 4 bits wide

5(b) Error-Correcting Code (ECC) & Hamming Code

1. How ECC works

- **English:** ECC adds **redundant bits (parity bits)** to memory data to **detect and correct errors**.
- **Bangla:** ECC extra bits **যোগ করে**, যা ব্যবহার করে **1-bit বা 2-bit error detect এবং correct করা যায়**।

2. Hamming Code Procedure

- **Step 1:** Place parity bits at positions 1, 2, 4, 8...
- **Step 2:** Calculate parity bits such that **each parity bit checks a unique combination of data bits**
- **Step 3:** If a single bit error occurs, the combination of parity errors identifies the **exact bit in error**
- **Step 4:** Flip the wrong bit → error corrected

Bangla summary:

- Hamming code → detect + correct single-bit error
- Parity bits → error location **알려준다**

5(c) Elements of a Machine Instruction & Instruction Cycle Diagram

1. Elements of a Machine Instruction

1. **Opcode** – Operation to perform (ADD, SUB, LOAD)

2. **Operand(s)** – Data or memory/register addresses
 3. **Addressing Mode** – How to interpret operand
 4. **Optional Control Bits** – Condition codes, immediate flags
-

2. Instruction Cycle State Diagram

States:

1. **Fetch** – Get instruction from memory → IR
2. **Decode** – Determine opcode & operands
3. **Execute** – Perform operation in ALU / memory
4. **Store / Write Back** – Save result to destination

Diagram (simplified):

[Fetch] → [Decode] → [Execute] → [Store/Write Back] → [Fetch]...

Bangla summary:

- CPU continually repeats **Fetch → Decode → Execute → Store**
-

5(d) Nested Procedures & Shifts

1. Nested Procedures

- **English:** A procedure (function) that **calls another procedure** or even itself (recursion).
- **Bangla:** Nested procedure হলো **একটি function অন্য function কে call করে বা নিজেকেও call করতে পারে।**
- **Example:**

```
void A() { B(); }
```

```
void B() { ... }
```

2. Arithmetic vs Logical Shift

Feature	Arithmetic Shift	Logical Shift
Shifts bits left/right	Preserves sign bit (for signed numbers)	Fills with 0 (ignores sign)
Use	Signed arithmetic	Unsigned numbers, bit manipulation
Example	1010 (-6, 4-bit) >> 1 → 1101 (-3)	1010 >> 1 → 0101

Bangla summary:

- Arithmetic shift → sign preserved
- Logical shift → 0 দিয়ে fill

💡 Quick Exam Tip:

- $16M \times 4 \text{ DRAM} = 4M \text{ words} \times 4 \text{ bits} \rightarrow \text{row/column} + 4\text{-bit bus}$
- Hamming code → parity bits detect & correct errors
- Machine instruction → opcode + operand + addressing + optional
- Instruction cycle = Fetch → Decode → Execute → Store
- Nested procedure = function calling another
- Arithmetic shift = signed, logical shift = unsigned

ঠিক আছে! আমি তোমাকে সহজ বাংলা + ইংরেজি মিলিয়ে step by step বুঝাই।

6(a) Register Addressing & x86 Instruction Formats

1. Register Addressing

- **English:** Register addressing is when the **operand of an instruction is located in a CPU register** instead of memory.
- **Bangla:** Register addressing হলো যখন instruction এর data CPU এর internal register থেকে নেওয়া হয়, memory access লাগে না → faster।

Example: ADD AX, BX → both operands in registers.

2. x86 Instruction Formats

- **x86 instructions** can have variable formats:
 1. **One-byte opcode** – simplest instruction (NOP)
 2. **One-byte opcode + Register/Memory operand** (ADD AX, [1000H])
 3. **Opcode + ModR/M byte + Displacement/Immediate data**
 - **ModR/M byte** specifies **register/memory**
 - **Immediate data** – constant value
 - **Bangla summary:** x86 instruction format = opcode + operand + optional displacement/immediate
-

6(b) User-Visible Registers & CPU Structure

1. User-Visible Registers

- **English:** Registers that **programmers can directly access** via instructions.
- **Bangla:** CPU register যা **programmer instruction দিয়ে read/write করতে পারে।**

Examples:

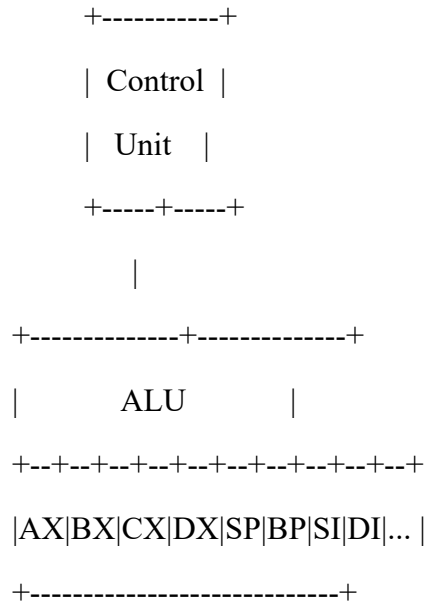
- General purpose: AX, BX, CX, DX (x86)
 - Index/Pointer: SI, DI, BP, SP
-

2. Internal Structure of CPU

Components:

1. **ALU** – arithmetic & logic operations
2. **Registers** – temporary storage (general + special)
3. **Control Unit (CU)** – fetch/decode/execute control
4. **Instruction Register (IR)** – stores current instruction
5. **Program Counter (PC)** – address of next instruction
6. **Memory Address Register (MAR)** – holds memory address
7. **Memory Buffer Register (MBR)** – data to/from memory

Diagram (simplified):



6(c) Control & Status Registers & Data Flow

1. Control Registers

- **English:** Registers that control CPU operation.
- **Bangla:** CPU এর behavior নিয়ন্ত্রণ করে।
- **Examples:**
 - **PC (Program Counter)** – next instruction address
 - **IR (Instruction Register)** – current instruction
 - **MAR / MBR** – memory access
 - **Status Register / Flags** – zero, carry, sign

2. Status Registers

- Contain **flags** that show results of operations:
 - Zero flag (Z), Carry flag (C), Sign flag (S), Overflow flag (O)

3. Data Flow of Fetch & Indirect Cycle

Fetch Cycle:

1. PC → MAR (address of instruction)

2. Memory $\rightarrow$ MBR $\rightarrow$ IR
3. PC $\leftarrow$ PC + 1

Indirect Cycle (if instruction uses memory operand):

1. MAR $\leftarrow$ effective address
2. Memory $\rightarrow$ MBR (load operand)

Bangla summary:

- Fetch $\rightarrow$ instruction memory থেকে IR এ
- Indirect $\rightarrow$ operand memory থেকে MBR এ

6(d) Instruction Pipelining & Six-Stage Pipeline

1. Instruction Pipelining

- **English:** Technique to **overlap execution of multiple instructions** in CPU stages to improve throughput.
- **Bangla:** CPU multiple instruction একসাথে বিভিন্ন stage এ process করে $\rightarrow$ faster execution।

2. Six-Stage Pipeline

Stage Function

IF Instruction Fetch
 ID Instruction Decode
 OF Operand Fetch
 EX Execute
 MA Memory Access
 WB Write Back

3. Timing Diagram (simplified)

Time $\rightarrow$

Instr1: IF ID OF EX MA WB

Instr2: IF ID OF EX MA WB

Instr3: IF ID OF EX MA WB

- Each instruction **overlaps in different stages**
- Throughput increases → execution faster

Bangla summary:

- Six stages → IF, ID, OF, EX, MA, WB
- Each instruction overlapped → pipeline efficiency

 Quick Tip Table:

Topic	Key Points
Register Addressing	Operand in CPU register → fast
x86 Format	Opcode + ModR/M + displacement/immediate
User-visible Register	AX, BX, CX, DX, SI, DI
Control/Status	PC, IR, MAR, MBR, Flags
Fetch Cycle	PC → MAR → MBR → IR → PC+1
Pipeline	6 stages → IF ID OF EX MA WB → overlapping execution
